

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Abstract Data Type”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 31 Maret 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR	ii
BAB I.....	4
PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat	5
BAB II.....	6
PEMBAHASAN	6
2.1 Pengertian ADT	6
2.2 Implementasi ADT pada Java	7
2.2.1 Class Inti (Jam_2511533016)	7
2.2.2 Class Driver 1 (JamDriver1_2511533016)	8
2.2.3 Class Driver 2 (JamDriver2_2511533016)	9
BAB III	12
PENUTUP.....	12
3.1 Kesimpulan	12
3.2 Saran	13
DAFTAR PUSTAKA	14
LAPORAN TUGAS PEKAN 1	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, pengelolaan data yang terstruktur dan efisien merupakan hal yang sangat penting. Salah satu konsep fundamental dalam struktur data yang mendukung hal tersebut adalah *Abstract Data Type* (ADT). ADT merupakan suatu model atau konsep yang mendefinisikan tipe data berdasarkan perilaku (operasi-operasi yang dapat dilakukan), tanpa harus menjelaskan secara rinci bagaimana data tersebut diimplementasikan di dalam memori.

Penggunaan ADT memungkinkan pemrogram untuk memisahkan antara *interface* (apa yang dapat dilakukan) dengan *implementation* (bagaimana cara melakukannya). Dengan demikian, program menjadi lebih modular, mudah dipahami, serta lebih fleksibel untuk dikembangkan atau dimodifikasi tanpa mempengaruhi bagian lain dari sistem. Hal ini sangat penting dalam pengembangan sistem yang kompleks.

Oleh karena itu, pemahaman mengenai ADT sangat penting bagi mahasiswa, khususnya dalam bidang struktur data dan pemrograman. Melalui praktikum ini, diharapkan mahasiswa dapat memahami konsep dasar ADT serta mampu mengimplementasikannya dalam berbagai kasus pemrograman.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 1 tentang ADT adalah sebagai berikut:

1. Memahami konsep dasar Abstract Data Type (ADT).
2. Mampu merancang ADT sesuai dengan kebutuhan permasalahan yang diberikan.

3. Mampu mengimplementasikan ADT ke dalam bahasa pemrograman (seperti Java) dengan menggunakan struktur data yang sesuai.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 1 tentang ADT adalah sebagai berikut:

1. Meningkatkan pemahaman mahasiswa mengenai konsep dasar ADT dalam pengelolaan data secara abstrak.
2. Menjadi dasar dalam pengembangan aplikasi yang lebih besar dan kompleks.
3. Melatih pola pikir logis dan sistematis dalam menyelesaikan permasalahan pemrograman.

BAB II

PEMBAHASAN

2.1 Pengertian ADT

Abstract Data Type (ADT) adalah suatu konsep dalam struktur data yang mendefinisikan tipe data berdasarkan perilaku atau operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa menjelaskan secara rinci bagaimana data tersebut diimplementasikan di dalam memori. Dengan kata lain, ADT berfokus pada *apa yang dilakukan* oleh suatu struktur data, bukan *bagaimana cara kerjanya*.

ADT memungkinkan pemrogram untuk memisahkan antara *interface* dan *implementation*. *Interface* berisi kumpulan operasi yang dapat dilakukan terhadap suatu data, seperti menambah, menghapus, atau mengakses data. Sedangkan *implementation* merupakan cara atau metode yang digunakan untuk merealisasikan operasi tersebut, misalnya menggunakan array, linked list, atau struktur lainnya.

Sebagai contoh, pada ADT *stack*, terdapat operasi seperti *push* (menambahkan data) dan *pop* (menghapus data). Namun, ADT tidak menentukan apakah *stack* tersebut diimplementasikan menggunakan array atau linked list. Hal ini memberikan fleksibilitas dalam pengembangan program serta memudahkan proses pemeliharaan dan pengembangan sistem.

Dengan menggunakan konsep ADT, program menjadi lebih terstruktur, modular, dan mudah dipahami, karena setiap bagian program memiliki tanggung jawab yang jelas dan terpisah.

2.2 Implementasi ADT pada Java

Pada praktikum pekan 1 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di papan tulis. Kode program yang dibuat tentang jam, di bawah ini kode program yang dituliskan pada praktikum pekan 1.

2.2.1 Class Inti (Jam_2511533016)

```
public final class Jam_2511533016 {
    private int hh;
    private int mm;
    private int ss;

    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    public Jam_2511533016(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    public int getHour() {return hh;}
    public int getMinute() {return mm;}
    public int getSeconds() {return ss;}

    public void setHour(int h) {this.hh = h;}
    public void setMinute(int m) {this.mm = m;}
    public void setSeconds(int s) {this.ss = s;}

    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511533016 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600;
        int h = total / 3600; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511533016(h, m, s);
    }

    public int compareTo(Jam_2511533016 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (! (o instanceof Jam_2511533016 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }

    public Jam_2511533016 plus(Jam_2511533016 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511533016 minus(Jam_2511533016 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
    public Jam_2511533016 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
    public Jam_2511533016 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
    public Jam_2511533016 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600)); }
    public Jam_2511533016 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600)); }

    public static int durasiDetik(Jam_2511533016 jaw, Jam_2511533016 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
}
```

Gambar 2.2.1

Program Java tersebut merupakan implementasi **Abstract Data Type (ADT) waktu (Jam)** yang merepresentasikan waktu dalam format **jam, menit, dan detik (hh:mm:ss)**. Kelas `Jam_2511533016` menggunakan prinsip **enkapsulasi**, di mana data disimpan dalam atribut `hh`, `mm`, dan `ss` yang bersifat *private*, sehingga tidak dapat diakses langsung dari luar kelas. Akses terhadap data hanya dapat dilakukan melalui method (fungsi), yang dalam konsep ADT disebut sebagai **operasi atau primitif**.

Kelas ini menyediakan berbagai operasi yang merepresentasikan perilaku dari ADT waktu. Terdapat method `isValid` untuk memvalidasi apakah nilai jam, menit, dan detik berada dalam rentang yang benar. Konstruktor digunakan untuk menginisialisasi objek waktu. Selain itu, terdapat *getter* dan *setter* yang berfungsi untuk mengambil dan mengubah nilai atribut, yang merupakan bentuk implementasi dari operasi dasar dalam ADT.

Program ini juga menyediakan fungsi konversi, yaitu `toSeconds` untuk mengubah waktu menjadi total detik, serta `fromSeconds` untuk membentuk kembali objek waktu dari jumlah detik tertentu. Proses ini memanfaatkan perhitungan matematis serta normalisasi waktu dalam satu hari (24 jam), sehingga nilai waktu tetap berada dalam batas yang valid.

Selain itu, terdapat berbagai operasi manipulasi waktu seperti plus dan minus untuk penjumlahan dan pengurangan waktu, serta `nextSecond`, `prevSecond`, dan variasinya untuk menambah atau mengurangi detik. Operasi ini menunjukkan bagaimana ADT dapat digunakan untuk melakukan perhitungan tanpa harus mengetahui detail penyimpanan data.

Kelas ini juga mendukung operasi perbandingan melalui method `compareTo`, yang membandingkan dua objek waktu berdasarkan total detik, serta `equals` untuk mengecek kesamaan nilai waktu. Method `hashCode` disediakan untuk mendukung penggunaan objek dalam struktur data berbasis hash. Selain itu, terdapat method `durasiDetik` untuk menghitung selisih waktu dalam satuan detik, serta `toString` untuk menampilkan waktu dalam format yang terstruktur dan mudah dibaca.

2.2.2 Class Driver 1 (JamDriver1_2511533016)

```
public class JamDriver_2511533016 {
    public static void main(String[] args) {
        Jam_2511533016 a = new Jam_2511533016(23, 59, 50);
        Jam_2511533016 b = new Jam_2511533016(0, 0, 15);
        System.out.println("a      =" + a);
        System.out.println("b      =" + b);
        System.out.println("a+b    =" + a.plus(b));
        System.out.println("next 20s =" + a.nextNSeconds(20));
        System.out.println("durasi(a,b) =" + Jam_2511533016.durasiDetik(a, b));
        System.out.println("a.compareTo(b) =" + a.compareTo(b));
    }
}
```

Gambar 2.2.2

Program tersebut merupakan **kelas driver (program utama)** yang digunakan untuk menguji implementasi **ADT waktu (Jam)** yang telah dibuat sebelumnya pada kelas `Jam_2511533016`. Di dalam method `main`, dibuat dua buah objek waktu yaitu `a` dengan nilai **23:59:50** dan `b` dengan nilai **00:00:15**. Kedua objek ini kemudian digunakan untuk melakukan berbagai operasi yang telah disediakan oleh ADT.

Program menampilkan nilai awal dari masing-masing objek menggunakan `System.out.println`, yang secara otomatis memanggil method `toString()` sehingga waktu ditampilkan dalam format **hh:mm:ss**. Selanjutnya dilakukan operasi penjumlahan waktu menggunakan method `plus`, di mana waktu `a` ditambahkan dengan waktu `b`, dan hasilnya ditampilkan. Program juga menguji operasi penambahan detik menggunakan `nextNSeconds(20)` yang menambahkan 20 detik ke waktu `a`.

Selain itu, program menghitung selisih waktu antara `a` dan `b` menggunakan method `durasiDetik`, yang menghasilkan perbedaan dalam satuan detik. Terakhir, dilakukan perbandingan antara kedua waktu menggunakan `compareTo`, yang akan menghasilkan nilai negatif, nol, atau positif tergantung apakah waktu `a` lebih kecil, sama, atau lebih besar dari `b`.

2.2.3 Class Driver 2 (JamDriver2_2511533016)

```
public class Jam_2511533016Driver2 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam ===");
        System.out.println("\n=== Input Jam 1 ===");
        Jam_2511533016 j1 = buatJamDariInput(input);

        System.out.println("\n=== Input Jam 2 ===");
        Jam_2511533016 j2 = buatJamDariInput(input);

        System.out.println("\n=== Hasil Operasi ===");
        System.out.println("Jam 1 (String)      : " + j1.toString());
        System.out.println("Jam 2 (String)      : " + j2.toString());
        System.out.println("Jam 1 Dalam detik   : " + j1.toSeconds());
        System.out.println("Jam 2 Dalam detik   : " + j2.toSeconds());

        int perbandingan = j1.compareTo(j2);
        if (perbandingan < 0) {
            System.out.println("Status      : Jam 1 lebih lambat (setelah) Jam 1");
        } else if (perbandingan < 0) {
            System.out.println("Status      : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status      : Jam 1 dan jam 2 sama persis");
        }

        System.out.println("Durasi (J1 ke J2)      : " + Jam_2511533016.durasiDetik(j1, j2) + "detik");

        Jam_2511533016 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik berikutnya : " + jNext);

        Jam_2511533016 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);

        Jam_2511533016 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2 : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai. ");
    }

    private static Jam_2511533016 buatJamDariInput(Scanner sc) {
        int h, m, s;
    }
}
```

```

        while (true) {
            System.out.println("Masukan Jam (0-23)    : ");
            h = sc.nextInt();
            System.out.println("Masukan Menit (0-59)    : ");
            m = sc.nextInt();
            System.out.println("Masukan Detik (0-59)    : ");
            s = sc.nextInt();

            if (Jam_2511533016.isValid(h, m, s)) {
                return new Jam_2511533016(h, m, s);
            } else {
                System.out.println("(Error) Input tidak valid ! Silahkan ulangi.\n");
            }
        }
    }
}

```

Gambar 2.2.3

Program tersebut merupakan **driver lanjutan** yang digunakan untuk menguji dan mengoperasikan **ADT waktu (Jam)** secara lebih interaktif dengan melibatkan input dari pengguna. Kelas `Jam_2511533016Driver2` berfungsi sebagai penghubung antara pengguna dan implementasi ADT, sehingga pengguna dapat langsung memasukkan nilai waktu dan melihat hasil dari berbagai operasi yang tersedia.

Pada method `main`, program terlebih dahulu membuat objek `Scanner` untuk membaca input dari keyboard. Selanjutnya, pengguna diminta untuk memasukkan dua buah waktu (Jam 1 dan Jam 2) melalui pemanggilan method `buatJamDariInput`. Method ini bertugas membaca input jam, menit, dan detik dari pengguna, kemudian melakukan validasi menggunakan method `isValid` dari kelas ADT. Jika input tidak valid, program akan meminta pengguna untuk mengulang hingga nilai yang dimasukkan benar. Hal ini menunjukkan penerapan kontrol input yang baik dalam penggunaan ADT.

Setelah kedua objek waktu berhasil dibuat, program menampilkan hasil dalam beberapa bentuk. Pertama, waktu ditampilkan dalam format string menggunakan `toString`, kemudian dikonversi ke dalam satuan detik menggunakan `toSeconds`. Selanjutnya dilakukan perbandingan antara Jam 1 dan Jam 2 menggunakan `compareTo`, yang hasilnya digunakan dalam percabangan `if-else` untuk menentukan apakah waktu pertama lebih awal, lebih lambat, atau sama dengan waktu kedua.

Program juga menghitung durasi antara kedua waktu menggunakan method `durasiDetik`, yang menghasilkan selisih dalam satuan detik. Selain itu, dilakukan operasi manipulasi waktu seperti mencari detik berikutnya (`nextSecond`) dan detik sebelumnya (`prevSecond`) dari Jam 1. Program juga melakukan penjumlahan dua waktu menggunakan method `plus`, dan hasilnya ditampilkan kepada pengguna.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Abstract Data Type* (ADT) merupakan konsep penting dalam struktur data yang mendefinisikan tipe data berdasarkan operasi atau perilakunya tanpa bergantung pada bagaimana data tersebut diimplementasikan. Dalam konteks pemrograman, operasi-operasi (primitif) pada ADT diwujudkan dalam bentuk method atau fungsi, sehingga pengguna hanya berinteraksi melalui operasi yang tersedia.

Implementasi ADT waktu (Jam) dalam program Java menunjukkan penerapan prinsip-prinsip utama ADT seperti enkapsulasi, abstraksi, dan modularitas. Kelas `Jam_2511533016` menyediakan berbagai operasi seperti validasi, konversi ke detik, manipulasi waktu (penjumlahan, pengurangan, penambahan detik), serta perbandingan waktu. Sementara itu, kelas driver digunakan untuk menguji dan mendemonstrasikan penggunaan ADT, baik secara langsung maupun melalui input pengguna.

Dengan adanya pemisahan antara *interface* dan *implementation*, program menjadi lebih terstruktur, mudah dipahami, serta lebih fleksibel untuk dikembangkan. Pengguna tidak perlu mengetahui detail internal dari pengolahan data, cukup menggunakan method yang telah disediakan. Hal ini membuktikan bahwa ADT sangat membantu dalam perancangan program yang efisien dan terorganisir.

3.2 Saran

Untuk praktikum selanjutnya, diharapkan materi dapat dikembangkan dengan studi kasus ADT yang lebih bervariasi agar mahasiswa semakin terlatih dalam menerapkan konsep abstraksi data pada berbagai permasalahan. Bagi mahasiswa, disarankan untuk lebih aktif berdiskusi dengan asisten praktikum ketika menemui kendala, serta membiasakan diri mempelajari materi sebelum praktikum dimulai agar proses pembelajaran dapat berjalan lebih efektif dan maksimal.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson dan R. L. Rivest, “Introducing to Algorithms,” 2009. [Online].
- [2] M. T. Goodrich, R. Tamassia dan M. H. Goldwasser, “Data Struchtures and Algorithms in Java,” 2014. [Online].

LAPORAN TUGAS PEKAN 1

SOAL :

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

a) Class Inti (Mobil_2511533016)

```
public class Jam_2511533016Driver2 {  
    public static void main(String[] args) {  
        Scanner input = new Scanner(System.in);  
        System.out.println("==== Program Driver Objek Jam ===");  
        System.out.println("Input Jam 1 ===");  
        Jam_2511533016 j1 = buatJamDariInput(input);  
  
        System.out.println("Input Jam 2 ===");  
        Jam_2511533016 j2 = buatJamDariInput(input);  
  
        System.out.println("==== Hasil Operasi ===");  
        System.out.println("Jam 1 (String) : " + j1.toString());  
        System.out.println("Jam 2 (String) : " + j2.toString());  
        System.out.println("Jam 1 Dalam detik : " + j1.toSeconds());  
        System.out.println("Jam 2 Dalam detik : " + j2.toSeconds());  
  
        int perbandingan = j1.compareTo(j2);  
        if (perbandingan < 0) {  
            System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 1");  
        } else if (perbandingan > 0) {  
            System.out.println("Status : Jam 1 lebih awal (sebelum) Jam 2");  
        } else {  
            System.out.println("Status : Jam 1 dan Jam 2 sama persis");  
        }  
  
        System.out.println("Durasi (J1 ke J2) : " + Jam_2511533016.durasiDetik(j1, j2) + "detik");  
  
        Jam_2511533016 jNext = j1.nextSecond();  
        System.out.println("Jam 1 Detik berikutnya : " + jNext);  
  
        Jam_2511533016 jPrev = j1.prevSecond();  
        System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);  
  
        Jam_2511533016 jHasilPlus = j1.plus(j2);  
        System.out.println("Hasil J1 + J2 : " + jHasilPlus);  
  
        input.close();  
        System.out.println("\nProgram Selesai. ");  
    }  
  
    private static Jam_2511533016 buatJamDariInput(Scanner sc) {  
        int h, m, s;  
        while (true) {  
            System.out.println("Masukan Jam (0-23) : ");  
            h = sc.nextInt();  
            System.out.println("Masukan Menit (0-59) : ");  
            m = sc.nextInt();  
            System.out.println("Masukan Detik (0-59) : ");  
            s = sc.nextInt();  
  
            if (Jam_2511533016.isValid(h, m, s)) {  
                return new Jam_2511533016(h, m, s);  
            } else {  
                System.out.println("(Error) Input tidak valid ! Silahkan ulangi.\n");  
            }  
        }  
    }  
}
```

Gambar a

Program tersebut merupakan driver (program utama) yang digunakan untuk menguji dan memanfaatkan class `Jam_2511533016`, yaitu class yang merepresentasikan waktu dalam satuan jam, menit, dan detik. Saat dijalankan, program dimulai dari method `main`, lalu membuat objek `Scanner` untuk membaca input dari pengguna. Selanjutnya, program meminta pengguna memasukkan dua buah waktu (Jam 1 dan Jam 2) melalui method `buatJamDariInput`.

Method `buatJamDariInput` bekerja dengan perulangan `while(true)` yang terus meminta input jam, menit, dan detik sampai pengguna memberikan nilai yang valid. Validasi dilakukan menggunakan method `isValid(h, m, s)` dari class `Jam_2511533016`, yang memastikan bahwa jam berada pada rentang 0–23, serta menit dan detik pada rentang 0–59. Jika input valid, method ini langsung mengembalikan objek `Jam_2511533016`; jika tidak, pengguna diminta mengulang input.

Setelah dua objek waktu (`j1` dan `j2`) berhasil dibuat, program menampilkan berbagai informasi terkait keduanya. Pertama, waktu ditampilkan dalam bentuk string menggunakan method `toString()`, lalu masing-masing waktu dikonversi ke total detik menggunakan `toSeconds()`. Berikutnya, program membandingkan kedua waktu menggunakan method `compareTo(j2)` yang menghasilkan nilai negatif, nol, atau positif untuk menentukan apakah Jam 1 lebih awal, sama, atau lebih lambat dibanding Jam 2, dan hasilnya ditampilkan dalam bentuk status.

Program juga menghitung selisih waktu antara Jam 1 dan Jam 2 dalam satuan detik melalui method `static durasiDetik(j1, j2)`. Selain itu, dilakukan beberapa operasi manipulasi waktu terhadap objek `j1`, yaitu mencari detik berikutnya dengan `nextSecond()`, detik sebelumnya dengan `prevSecond()`, serta penjumlahan dua waktu menggunakan `plus(j2)` yang menghasilkan objek waktu baru.

Di akhir eksekusi, `Scanner` ditutup dan program menampilkan pesan bahwa program selesai. Secara keseluruhan, program ini berfungsi sebagai alat uji yang lengkap untuk berbagai operasi pada objek waktu, mulai dari input, validasi, konversi, perbandingan, hingga manipulasi waktu.

b) Class Drive (DriveMobil_2511533016)

```
import java.util.ArrayList;

public class DriveMobil_2511533016 {

    public static void main(String[] args) {
        ArrayList<Mobil_2511533016> daftarMobil = new ArrayList<>();

        // Tambah mobil
        Mobil_2511533016 m1 = new Mobil_2511533016("Avanza", 2020, 1300, 150000000, "Toyota");
        Mobil_2511533016 m2 = new Mobil_2511533016("Brio", 2022, 1200, 170000000, "Honda");

        daftarMobil.add(m1);
        daftarMobil.add(m2);

        System.out.println("=== Data Mobil ===");
        for (Mobil_2511533016 m : daftarMobil) {
            m.tampil();
        }

        // Hapus mobil (contoh hapus index ke-0)
        daftarMobil.remove(0);

        System.out.println("=== Setelah Hapus ===");
        for (Mobil_2511533016 m : daftarMobil) {
            m.tampil();
        }

        // Contoh penggunaan setter (mutator)
        m2.setHarga(180000000);

        System.out.println("=== Setelah Update Harga ===");
        m2.tampil();
    }
}
```

Gambar b

Program tersebut merupakan driver (program utama) yang digunakan untuk mengelola dan menguji class `Mobil_2511533016`, yaitu class yang merepresentasikan data sebuah mobil (seperti nama, tahun, kapasitas mesin, harga, dan merek). Saat dijalankan, program dimulai dari method `main`, lalu membuat sebuah `ArrayList` bernama `daftarMobil` yang berfungsi untuk menyimpan banyak objek mobil dalam bentuk list (daftar dinamis).

Selanjutnya, program membuat dua objek mobil yaitu `m1` dan `m2` dengan memanggil constructor `Mobil_2511533016` dan mengisi atributnya secara langsung (contohnya: nama mobil, tahun produksi, kapasitas mesin, harga, dan merek). Setelah itu, kedua objek tersebut dimasukkan ke dalam `ArrayList` menggunakan method `add()`, sehingga sekarang daftar mobil berisi dua data.

Program kemudian menampilkan seluruh data mobil dengan melakukan perulangan `for-each`, di mana setiap objek mobil dalam `daftarMobil` dipanggil method `tampil()` untuk mencetak informasi mobil ke layar. Setelah itu, program melakukan operasi penghapusan data, yaitu menghapus mobil pada indeks ke-0 menggunakan `remove(0)`. Artinya, mobil pertama dalam daftar akan dihapus, dan daftar akan tersisa satu mobil saja. Hasil setelah penghapusan ini kemudian ditampilkan kembali menggunakan perulangan yang sama untuk menunjukkan perubahan isi list.

Berikutnya, program menunjukkan penggunaan setter (mutator) dengan mengubah harga mobil kedua (m2) melalui method `setHarga(1800000000)`. Ini berarti nilai harga pada objek tersebut diperbarui tanpa membuat objek baru. Setelah harga diubah, program kembali menampilkan data mobil tersebut menggunakan method `tampil()` untuk memperlihatkan hasil perubahan.

Di akhir program, tidak ada input dari pengguna karena semua data sudah diinisialisasi langsung di dalam kode. Secara keseluruhan, program ini berfungsi untuk демонстраци penggunaan `ArrayList` dalam menyimpan objek, serta operasi dasar pada objek seperti menambah data, menampilkan data, menghapus data, dan mengubah nilai atribut menggunakan setter.